

+ MyWalk / Tribute

Memorization Module

Full Implementation Plan for Claude Code

Document Scope

This document is a complete, handoff-ready implementation plan for a professional-grade scripture and text memorization module inside the MyWalk (Tribute) Flutter application. It covers:

- Full data architecture — Firestore schema with field-level detail
- SM2 spaced repetition algorithm — complete mathematical derivation and Flutter implementation
- All five review modes — Cloze, Progressive Recall, Flip Card, Typing, and Recitation — with UI specifications
- Initial memorization flow — multi-step onboarding for new items
- Claude API integration — chunking prompt design and error handling
- Analytics and community features
- Claude Code task breakdown — sprint-by-sprint, file-by-file
- Notification architecture and haptic feedback patterns

This plan assumes the existing MyWalk Flutter/Firebase/Firestore project structure. All file paths are relative to the project root. Review modes are implemented as independent widgets within a shared `MemorizationReviewShell`.

1. Data Architecture

1.1 Firestore Collections

All memorization data lives under the user's document. This keeps security rules simple and enables offline-first caching.

Firestore Path	Purpose
/users/{uid}/memorizations/{itemId}	Root document for each memorization item
/users/{uid}/memorizations/{itemId}/attempts/{attemptId}	Individual review attempt records (sub-collection for scalability)
/users/{uid}/memorizations/{itemId}/shares/{shareId}	Friends granted read access to this item's progress
/memorizationCircles/{circleId}	Community memorization circles (top-level, shared across users)

1.2 MemorizationItem Document

Complete field specification for the root document:

Field	Type	Description
id	String	Firestore document ID (auto-generated)
userId	String	Owner's Firebase UID
title	String	Human-readable label, e.g. "John 3:16" or "Westminster Shorter Q1"
fullText	String	Complete verbatim text to be memorized
chunks	List<Map>	AI-generated phrase chunks — see TextChunk schema below
status	String enum	"active" "mastered" "archived"
habitSubCategoryId	String?	Optional link to MyWalk habit sub-category (e.g. scriptureRecall)
createdAt	Timestamp	Firestore server timestamp at creation
lastReviewedAt	Timestamp?	Timestamp of most recent completed review session
nextReviewDate	Timestamp	Computed by SM2 — when next review is due
intervalDays	double	Current SM2 interval in days (starts at 1)

easeFactor	double	SM2 ease factor — starts at 2.5, min 1.3
repetitionCount	int	Number of successful consecutive reviews (SM2 n)
totalAttempts	int	Lifetime review count across all modes
successfulAttempts	int	Attempts rated quality >= 3
masteryPercent	double	$(\text{successfulAttempts} / \text{totalAttempts}) \times 100$
audioUrl	String?	Cloud Storage URL of user's own voice recording
ttsUrl	String?	Cloud Storage URL of TTS-generated audio (cached)
streakCount	int	Consecutive days reviewed successfully

1.3 TextChunk Schema (embedded in chunks array)

Field	Type	Description
id	String	UUID generated client-side
sequenceNumber	int	Display order (0-indexed)
text	String	The actual phrase, e.g. "For God so loved the world"
hint	String	First-letter scaffold, e.g. "F G s l t w"
wordCount	int	Pre-computed word count for difficulty weighting
successCount	int	Times recalled correctly across all reviews
attemptCount	int	Times this chunk was specifically tested
strengthScore	double	$\text{successCount} / \text{attemptCount}$ — drives heat map colour

1.4 ReviewAttempt Schema (sub-collection)

Field	Type	Description
mode	String	"cloze" "progressive" "flipCard" "typing" "recitation"
attemptedAt	Timestamp	Server timestamp
qualityScore	int (0-5)	SM2 quality: 0=blackout, 1=wrong but correct on hint, 2=wrong but easy, 3=correct with difficulty, 4=correct with pause, 5=perfect
confidence	int (1-5)	User-reported confidence rating
timeToRecallSeconds	int	Wall-clock time from prompt to submission
missedChunkIds	List<String>	IDs of chunks that failed — drives heat map update

userResponse	String?	Raw typed response (typing mode only, for debug/review)
levenshteinScore	double?	Similarity ratio 0–1 for typing mode (1 = perfect)
nextReviewScheduledFor	Timestamp	Computed SM2 next review date stored with the attempt for audit trail
intervalDaysApplied	double	Interval that was applied after this review
easeFactorAfter	double	EF value stored after this review for audit trail

2. SM2 Spaced Repetition — Complete Derivation

2.1 Algorithm Overview

The SM2 algorithm (SuperMemo 2, Wozniak 1987) is the foundation of Anki and most modern spaced repetition systems. It computes an optimal review interval based on how well you recalled material, updating two state variables:

State variables per item:

- intervalDays (I) — days until next review. Starts at 1.
- easeFactor (EF) — multiplier controlling how fast intervals grow. Starts at 2.5, minimum 1.3.
- repetitionCount (n) — how many consecutive successful reviews. Resets on failure.

2.2 Quality Score (q) — The Input

Each review produces a quality score $q \in \{0,1,2,3,4,5\}$:

q	Rating	Meaning
0	Complete blackout	No recollection at all
1	Wrong — hint needed	Incorrect, recalled correctly only after seeing answer
2	Wrong — easy recall	Incorrect, but felt easy once shown
3	Correct with difficulty	Recalled correctly but took significant effort
4	Correct with pause	Recalled correctly with slight hesitation
5	Perfect recall	Instant, effortless, correct recall

In MyWalk, q is derived from two user inputs: the confidence slider (1–5) and the objective accuracy of their response. See §2.5 for the mapping formula.

2.3 SM2 Update Formulas

For each completed review, apply these rules in order:

Step 1: Update Ease Factor

$$EF' = EF + (0.1 - (5 - q) \times (0.08 + (5 - q) \times 0.02))$$

Simplified: $EF' = EF - 0.8 + 0.28q - 0.02q^2$

Then clamp: $EF' = \max(1.3, EF')$

q value	EF change	Example (EF=2.5)	New EF	Effect
0	-0.80	2.5 - 0.80	1.70 (or 1.3 min)	Major slowdown
1	-0.54	2.5 - 0.54	1.96	Slowdown
2	-0.32	2.5 - 0.32	2.18	Slight slowdown
3	-0.14	2.5 - 0.14	2.36	Near neutral
4	+0.00	2.5 + 0.00	2.50	Neutral
5	+0.10	2.5 + 0.10	2.60	Speeds up

Step 2: Update Interval

Apply the interval rules:

```

if q < 3:
    // Failed review — reset learning
    n = 0
    I = 1 // Review again tomorrow

else:
    if n == 0: I = 1 // First successful review
    elif n == 1: I = 6 // Second success: ~1 week
    else: I = round(I × EF) // Subsequent: exponential growth
    n += 1

nextReviewDate = today + I days

```

Step 3: Grace-Aligned Adjustments (MyWalk Extension)

Standard SM2 has no grace period. MyWalk adds two adjustments aligned with a grace-not-performance theology:

- Late review forgiveness: If a review is completed 1–3 days late, apply it as if on time (no interval penalty). Beyond 3 days late, reduce I by 20% but do not reset to 0.
- Streak bonus: If streakCount ≥ 7 , add 1 day to the computed interval as a gentle encouragement reward.

2.4 Flutter Implementation

Create lib/features/memorization/services/sm2_service.dart:

```
class SM2Service {
```

```
static const double minEaseFactor = 1.3;
static const double initialEaseFactor = 2.5;

static SM2Result computeNextReview({
    required int qualityScore,          // 0-5
    required double currentEF,
    required double currentInterval, // days
    required int repetitionCount,
    required int streakCount,
    required DateTime lastReviewDue, // when it was scheduled
    required DateTime reviewedAt,    // when user actually reviewed
}) {
    // Step 1: Grace period adjustment
    final daysLate = reviewedAt.difference(lastReviewDue).inDays;
    double intervalMultiplier = 1.0;
    if (daysLate > 3) intervalMultiplier = 0.8;

    // Step 2: Update ease factor
    double newEF = currentEF + (0.1 - (5 - qualityScore) *
        (0.08 + (5 - qualityScore) * 0.02));
    newEF = newEF.clamp(minEaseFactor, 5.0);

    // Step 3: Compute new interval
    double newInterval;
    int newN;

    if (qualityScore < 3) {
        newInterval = 1;
        newN = 0;
    } else {
        if (repetitionCount == 0) newInterval = 1;
        else if (repetitionCount == 1) newInterval = 6;
        else newInterval = (currentInterval * newEF *
intervalMultiplier).roundToDouble();
        newN = repetitionCount + 1;
        // Streak bonus
        if (streakCount >= 7) newInterval += 1;
    }

    final nextDate = DateTime.now().add(Duration(days:
newInterval.round()));
    return SM2Result(
        newEaseFactor: newEF,
        newInterval: newInterval,
        newRepetitionCount: newN,
        nextReviewDate: nextDate,
    );
}
```

2.5 Quality Score Derivation Formula

Map user inputs to SM2 quality score:

Objective Accuracy	Confidence (user slider)	→ q
< 50% correct (cloze/typing)	Any	0 or 1
50–79% correct	1–2	2
50–79% correct	3–5	3
80–94% correct	1–3	3
80–94% correct	4–5	4
95–100% correct	1–3	4
95–100% correct	4–5	5
Flip card: "I knew it"	User-selected	confidence–1+2 capped 5
Flip card: "Didn't know"	Any	1

3. Review Modes — Full Specification

3.1 Shared Architecture: MemorizationReviewShell

All five review modes are loaded inside a shared shell widget that handles:

- Session timer (starts on first interaction, stops on submit)
- Progress bar (chunk index / total chunks)
- Confidence slider (shown after each chunk)
- Haptic feedback triggers (light on correct, heavy on error)
- SM2 update on session completion
- Transition animation between modes

File: lib/features/memorization/widgets/memorization_review_shell.dart

```
class MemorizationReviewShell extends StatefulWidget {  
  final MemorizationItem item;  
  final ReviewMode mode;  
  final VoidCallback onComplete;  
}
```

3.2 Mode 1: Cloze Test

Behaviour

- Show full text with 1–3 randomly selected words replaced by underlined blanks
- User taps a blank → inline keyboard appears at bottom of screen
- Correct: blank fills green, haptic light tick
- Incorrect: blank fills red, correct word shown, haptic double-bump
- After all blanks: confidence slider → score computed → next chunk

Difficulty Scaling

- Attempt 1: 1 blank per chunk (easiest)
- Attempts 2–4: 2 blanks per chunk
- Attempt 5+: 3 blanks per chunk OR full chunk blank

Blank Selection Logic

Select "meaningful" words (avoid: the, a, an, is, was, and, or, but, of, in, to). Prefer nouns, verbs, and adjectives. Use NLTK-style stop word list baked into Dart code as a constant `Set<String>`.

Accuracy Calculation

$\text{accuracy} = \text{correctBlanks} / \text{totalBlanks}$ — fed into quality score formula (§2.5).

Key Widget

```
lib/features/memorization/widgets/modes/cloze_mode_widget.dart
```

3.3 Mode 2: Progressive Recall

Three-Step Flow (per chunk)

- Step 1 — First-letter scaffold: "F G s l t w" with remaining letters blanked
- Step 2 — First 3 words shown: "For God so ____" — rest blanked
- Step 3 — Blank: prompt shows only chunk number, user types or recites freely

Navigation

User taps "Next Step" button to advance. Can tap "Reveal" at any step (marks as q=2 for that chunk). Steps shown in a top indicator: ○●○ style dots.

Hint Generation

```
String generateHint(String text) {  
  return text.split(' ').map((w) {  
    if (w.isEmpty) return '';  
    return w[0] + '_' * (w.length - 1);  
  }).join(' ');  
}
```

Key Widget

```
lib/features/memorization/widgets/modes/progressive_recall_widget.dart
```

3.4 Mode 3: Flip Card

Behaviour

- Card shows chunk text on front
- User taps card → 3D flip animation → back shows reference + next chunk preview
- Two buttons: "I knew it ✓" (haptic success) and "I didn't know X" (haptic error)
- Stack of cards; swipe right = knew, swipe left = didn't know (optional gesture)

Animation

Use AnimationController with TweenAnimationBuilder rotating on Y-axis. Duration: 400ms, curve: Curves.easeInOut.

Quality Mapping

"Knew it" → use confidence slider value mapped to q 3–5. "Didn't know" → q = 1.

Key Widget

```
lib/features/memorization/widgets/modes/flip_card_widget.dart
```

3.5 Mode 4: Typing Mode

Behaviour

- Prompt: "Type the full memorization from memory"
- Large multiline TextField with no autocorrect, no autocapitalize
- Word count shown as user types: "47 / 78 words"
- Submit button enabled when word count $\geq$ 50% of target

Comparison Algorithm

Use Levenshtein distance normalised to a similarity ratio 0–1:

```
double levenshteinSimilarity(String a, String b) {  
  // Normalise: lowercase, collapse whitespace, strip punctuation  
  final na = _normalise(a); final nb = _normalise(b);  
  final dist = _levenshtein(na, nb);  
  final maxLen = max(na.length, nb.length);  
  return maxLen == 0 ? 1.0 : 1.0 - (dist / maxLen);  
}
```

similarity $\geq$ 0.95 → perfect. 0.80–0.94 → good. 0.60–0.79 → fair. $<$ 0.60 → poor.

Diff Highlighting

After submission, show side-by-side: user's text on left, correct text on right. Mismatched words highlighted in red on user side, green on correct side.

Key Widget

```
lib/features/memorization/widgets/modes/typing_mode_widget.dart
```

3.6 Mode 5: Recitation (Audio)

Behaviour

- Show full text dimmed in background
- Large microphone button: hold to record, release to stop
- Waveform visualiser during recording (use flutter_sound)
- On release: submit to Whisper API (or on-device if available) for transcription
- Compare transcription to fullText using Levenshtein similarity
- Show diff highlighting (same as Typing Mode)

Whisper Integration

```
// In Cloud Functions or direct from app  
final transcription = await WhisperService.transcribe(  
  audioPath: localFilePath,  
  language: 'en',  
  prompt: item.title, // helps Whisper with scripture names  
);
```

Fallback

If network unavailable, store the recording and mark attempt as "pending transcription". Retry on next app open.

Key Widget

```
lib/features/memorization/widgets/modes/recitation_mode_widget.dart
```

4. Initial Memorization Flow

4.1 New Item Creation (3-Screen Flow)

Screen 1: Input

- TextField for title (e.g. "John 3:16")
- Large TextField for full text (paste or type)
- Optional: link to habit sub-category via DropdownButtonFormField
- CTA: "Prepare for Memorization →"
- On tap: call Claude API (chunking) — show skeleton loader while waiting

Screen 2: Review Chunks

- Display returned chunks as reorderable cards (ReorderableListView)
- Each card shows chunk text + generated hint
- Tap to edit text inline
- Drag handle to reorder
- "Add chunk" button (max 8 chunks)
- "Remove" swipe gesture
- CTA: "Begin Initial Memorization →"

Screen 3: Multi-Step Initial Session

This is the first deep encounter with the material. Run in order:

- Step 1 — Listen: Play TTS audio of each chunk. Show large text. Button: "Play Again" and "Next Chunk →"
- Step 2 — Read Aloud: User reads each chunk aloud into mic. Record with flutter_sound. Haptic reward on completion.
- Step 3 — Visual Snapshot: Show all chunks stacked in full-paragraph view. "I have the shape of it" button.
- Step 4 — First Quick Test: Run one pass of Flip Card mode over all chunks.
- Complete: celebration screen → schedule first review for 12 hours later.

4.2 Claude API Chunking Prompt

POST to /v1/messages with:

```
system: "You are a scripture memorization expert. Your job is to split text into 2-5 natural phrase chunks for memorization. Each chunk should be a complete syntactic unit (clause or phrase), 5-12 words. Return
```

```
ONLY a JSON array. No preamble. Format:  
[{"text": "...", "hint": "F__ G__ s_ ..."}]  
where hint replaces each word with its first letter + underscores."  
  
user: "Split this into memorization chunks:\n{fullText}"
```

Always send request asynchronously. If Claude API fails, fall back to splitting on punctuation (.,,:;) or every 8 words.

4.3 TTS Audio Generation

Use Google Cloud Text-to-Speech (Neural2 voice en-US-Neural2-J for male, en-US-Neural2-F for female — user preference). Cache audio file to Cloud Storage under:

```
gs://mywalk-tts/{userId}/{itemId}/{chunkId}.mp3
```

On first load, generate all chunk audios in parallel via Cloud Function. Store ttsUrl in each chunk map after generation.

5. Analytics Dashboard & Community

5.1 Per-Item Dashboard

Accessed via tap on any item in the memorization list. Shows:

Widget	Data Source & Display
Mastery Ring	CircularProgressIndicator styled ring showing masteryPercent. Colour: red < 50%, amber 50–79%, green 80–100%.
Next Review	Days/hours until nextReviewDate. Green if future, red if overdue.
Chunk Heat Map	Grid of chunk cards. Background colour based on strengthScore: red 0–0.4, amber 0.4–0.7, green 0.7–1.0.
Attempt Timeline	Horizontal scrollable list of attempt icons. Tap to see details (mode, score, time).
Mode Distribution	Pie chart (fl_chart) showing which modes have been used most.
Time Invested	Sum of timeToRecallSeconds across all attempts, displayed as "X mins memorized".
Streak Badge	Flame icon + streakCount. Special colour/animation at 7, 30, 100 days.

5.2 Global Memorization Dashboard

Accessible from the main MyWalk nav. Summarises across all items:

- Items active / mastered / archived
- Due today (sorted by urgency)
- Weekly review activity chart (bar chart, fl_chart)
- Total words memorized (sum of wordCount across mastered items)
- Recent community activity

5.3 Community Features

Sharing an Item

- Long-press item → Share → Search for friend by username or email
- Creates /memorizations/{itemId}/shares/{friendUid} document with accessLevel: "read_only"
- Friend receives notification and can see item in their "Shared With Me" section
- Friend sees: chunks (read-only), your mastery %, your review streak, your attempt timeline
- Friend cannot see raw attempt data (privacy)

Memorization Circles

- Users can create or join a Circle (similar to Prayer Circle feature)
- Shared memorization goal: e.g. "Psalms 23 by Christmas"
- Each member sees a leaderboard of mastery % across the shared item
- Circle leader can set the item text and target date
- Daily notification: "3 of 5 members reviewed today — be one of them!"

Accountability Comments

- Comment thread per shared item (same UI as Prayer Circle comments)
- Reaction emoji limited to † 🙏 ❤️ 💧 — curated for context
- Prayer request tag: "Pray for me to remember this"

6. Notifications & Haptics

6.1 Notification Types

Notification	Trigger	Copy (examples)
Review Due	1 hr before nextReviewDate	"Time to revisit John 3:16 — your next review is ready 📖"
Review Overdue	24 hrs after nextReviewDate	"Your John 3:16 review was due yesterday. A few minutes now keeps it fresh."
Streak Milestone	streakCount = 7, 30, 100	"7-day streak! You're building something lasting. ✨"
Mastery Achieved	masteryPercent crosses 90%	"John 3:16 is yours. Hidden in your heart. 🌿"
Circle Nudge	If user hasn't reviewed while others have	"Sarah reviewed Psalm 23 today — stay with the circle."
Friend Shared	New share document created	"James shared Romans 8:28 with you — they'd love for you to join them."

All notifications are opt-in, configurable per item in item settings. Use `firebase_messaging` for remote push and `flutter_local_notifications` for scheduled local alerts.

6.2 Haptic Feedback Patterns

Event	Haptic Pattern	Flutter API Call
Correct answer (cloze/typing)	Light single tap	<code>HapticFeedback.lightImpact()</code>
Incorrect answer	Double medium bump	<code>HapticFeedback.mediumImpact() × 2</code>
Chunk completed	Medium single	<code>HapticFeedback.mediumImpact()</code>
Session complete	Success pattern (3 rapid lights)	Custom: 3× <code>lightImpact</code> , 60ms apart
Mastery milestone	Heavy + vibrate	<code>HapticFeedback.heavyImpact()</code> + <code>Vibration.vibrate(duration: 200)</code>
Card flip	Selection click	<code>HapticFeedback.selectionClick()</code>

7. Claude Code Implementation Breakdown

Sprint-by-sprint task list. Each sprint should be handed to Claude Code as a separate session with the relevant files listed.

SPRINT 1 — Data Layer & State Management

#	Task	Files
1	Create MemorizationItem model + fromFirestore / toFirestore	lib/features/memorization/models/memorization_item.dart
2	Create TextChunk model	lib/features/memorization/models/text_chunk.dart
3	Create ReviewAttempt model	lib/features/memorization/models/review_attempt.dart
4	Implement SM2Service with full formula and grace period extension	lib/features/memorization/services/sm2_service.dart
5	Create MemorizationRepository (Firestore CRUD + stream)	lib/features/memorization/repositories/memorization_repository.dart
6	Create MemorizationCubit + MemorizationState with Bloc	lib/features/memorization/cubit/memorization_cubit.dart
7	Unit tests for SM2Service (all q values, grace period, streak bonus)	test/features/memorization/sm2_service_test.dart

SPRINT 2 — Claude API Chunking & TTS

#	Task	Files
8	Implement ClaudeChunkingService with prompt from §4.2 + JSON parse + fallback	lib/features/memorization/services/chunking_service.dart
9	Implement hint generator (first-letter scaffold)	lib/features/memorization/utils/hint_generator.dart
10	Cloud Function: generateTTS — accepts itemId, generates MP3 per chunk to Cloud Storage	functions/src/generateTts.ts
11	TTSService in Flutter — trigger Cloud Function, cache URLs, AudioPlayer wrapper	lib/features/memorization/services/tts_service.dart

12	RecordingService — flutter_sound wrapper for user voice recording + Cloud Storage upload	lib/features/memorization/services/recording_service.dart
----	--	---

SPRINT 3 — New Item Creation Flow

#	Task	Files
13	MemorizationInputScreen — title + text input + habit link + skeleton loader	lib/features/memorization/screens/memorization_input_screen.dart
14	ChunkReviewScreen — ReorderableListView of chunks, edit/add/remove	lib/features/memorization/screens/chunk_review_screen.dart
15	InitialMemorizationSession — 4-step flow (Listen, Read Aloud, Visual, Flip)	lib/features/memorization/screens/initial_memorization_screen.dart
16	Celebration screen + confetti animation + "First review in 12 hours" card	lib/features/memorization/widgets/memorization_celebration.dart

SPRINT 4 — Five Review Mode Widgets

#	Task	Files
17	MemorizationReviewShell — timer, progress bar, confidence slider, haptics	lib/features/memorization/widgets/memorization_review_shell.dart
18	ClozeMode — blank selection logic, inline keyboard, diff rendering	lib/features/memorization/widgets/modes/cloze_mode_widget.dart
19	ProgressiveRecallMode — 3-step flow, hint rendering, step dots	lib/features/memorization/widgets/modes/progressive_recall_widget.dart
20	FlipCardMode — 3D flip animation, swipe gestures, knew/didn't know buttons	lib/features/memorization/widgets/modes/flip_card_widget.dart
21	TypingMode — multiline field, Levenshtein comparison, diff highlight	lib/features/memorization/widgets/modes/typing_mode_widget.dart

22	RecitationMode — mic button, waveform, Whisper transcription, diff highlight	lib/features/memorization/widgets/modes/recitation_mode_widget.dart
23	Mode selection screen — user picks which mode for this session	lib/features/memorization/screens/mode_selection_screen.dart

SPRINT 5 — Analytics Dashboard

#	Task	Files
24	MemorizationListScreen — sorted by dueDate, mastery ring, streak badge	lib/features/memorization/screens/memorization_list_screen.dart
25	ItemDashboardScreen — mastery ring, heat map, attempt timeline, mode pie	lib/features/memorization/screens/item_dashboard_screen.dart
26	ChunkHeatMap widget — colour-coded chunk grid with strengthScore	lib/features/memorization/widgets/chunk_heat_map.dart
27	AttemptTimeline widget — horizontal scrollable attempt history	lib/features/memorization/widgets/attempt_timeline.dart
28	GlobalMemorizationDashboard — aggregate stats, weekly bar chart, due list	lib/features/memorization/screens/global_dashboard_screen.dart

SPRINT 6 — Notifications, Community & Polish

#	Task	Files
29	Cloud Function: scheduleReviewNotification — triggered on nextReviewDate write	functions/src/scheduleReviewNotification.ts
30	NotificationService in Flutter — local + remote, per-item opt-in settings	lib/features/memorization/services/notification_service.dart
31	SharingService — create/revoke share documents, friend search	lib/features/memorization/services/sharing_service.dart
32	SharedWithMeScreen — read-only view of friend's item progress	lib/features/memorization/screens/shared_with_me_screen.dart
33	MemorizationCircle screens — create, join, leaderboard, comment thread	lib/features/memorization/screens/circle_screens/

34	Achievement badges — first mastery, 7/30/100 streaks, 10 items active	lib/features/memorization/widgets/achievement_badge.dart
35	Full integration testing + Firestore security rules for memorization collections	firestore.rules, test/integration/

8. Dependencies & Configuration

8.1 Flutter pubspec.yaml additions

```
# Existing (assumed present)
flutter_bloc: ^8.x
cloud_firestore: ^4.x
firebase_storage: ^11.x
firebase_messaging: ^14.x

# New additions for memorization module
flutter_sound: ^9.x           # Audio recording + playback
audioplayers: ^5.x           # TTS playback
flutter_local_notifications: ^17.x # Scheduled review alerts
fl_chart: ^0.68.x            # Mastery ring + pie + bar charts
vibration: ^1.x              # Haptic patterns
string_similarity: ^2.x       # Levenshtein distance
reorderable_list: ^0.2.x     # Chunk reordering
confetti: ^0.7.x             # Celebration animations
http: ^1.x                   # Claude API + Whisper calls
```

8.2 Firestore Security Rules (additions)

```
match /users/{userId}/memorizations/{itemId} {
  allow read, write: if request.auth.uid == userId;
  // Shared access for friends
  allow read: if exists(/databases/$(database)/documents/users/$(userId)
    /memorizations/$(itemId)/shares/$(request.auth.uid));

  match /attempts/{attemptId} {
    allow read, write: if request.auth.uid == userId;
  }
  match /shares/{shareId} {
    allow read: if request.auth.uid == userId || request.auth.uid ==
shareId;
    allow write: if request.auth.uid == userId;
  }
}
match /memorizationCircles/{circleId} {
  allow read: if request.auth != null;
  allow write: if request.auth.uid == resource.data.createdBy
    || request.auth.uid in resource.data.memberIds;
}
```

8.3 Environment Variables

Variable	Value / Source
CLAUDE_API_KEY	Anthropic console — store in Cloud Secret Manager, not in app
GOOGLE_CLOUD_TTS_KEY	GCP console — Cloud Text-to-Speech API
OPENAI_WHISPER_KEY	OpenAI console — only needed if using cloud Whisper (vs on-device)
TTS_CLOUD_BUCKET	gs://mywalk-tts (create in GCP console)

Never embed API keys in Flutter app binary. Route all Claude API and TTS calls through Cloud Functions. Only Whisper transcription may be called directly from the app if using OpenAI API (acceptable risk — key exposed only to authenticated users).

9. Grace-Aligned UX Principles

Every design decision should reflect the theology of MyWalk: memorization as worship and meditation, not performance or achievement. The following UX principles enforce this:

9.1 Language

- "You're meditating on John 3:16" — not "You're learning John 3:16"
- "Hidden in your heart" — not "Mastered"
- "Keep it fresh" — not "Don't break your streak"
- "Today's review" — not "Due item"
- Achievements framed as gift, not merit: "Grace has worked this deep into you †"

9.2 Failure Handling

- No red banners, no shame UI on failed recall
- On low-quality review: "The Spirit intercedes where words fail — keep going"
- Reset intervals are invisible to the user; they only see "Next review: tomorrow"
- Late reviews show warm encouragement, not guilt: "Glad you're here. Whenever you can."

9.3 Celebration

- Mastery celebration uses warmly worded scripture: "Thy word have I hid in mine heart — Psalm 119:11"
- Confetti uses brand colours (gold + deep navy) not generic rainbow
- Celebration screen includes "Share this milestone" button → share to Prayer Circle or external

9.4 Mode Naming in UI

Technical Mode	User-Facing Label	Icon
Cloze Test	Fill the Word	
Progressive Recall	Step by Step	
Flip Card	Meditate & Reveal	
Typing Mode	Write It Out	
Recitation	Speak It Aloud	

† End of Implementation Plan

MyWalk / Tribute — Memorization Module